Summary

- Sensors are a special type of operators that continuously poll for a given condition to be true.
- Airflow provides a collection of sensors for various systems/use cases; a custom condition can also be made with the `PythonSensor`.
- The `TriggerDagRunOperator` can trigger a DAG from another DAG, while the `ExternalTaskSensor` can poll the state in another DAG.
- Triggering DAGs is possible from outside Airflow with the REST API and/or CLI.

7 Communicating with external systems

This chapter covers

- Working with Airflow operators performing actions on systems outside Airflow
- Applying operators specific to external systems
- Implementing operators in Airflow doing A-to-B operations
- Testing tasks connecting to external systems

In all previous chapters, we've focused on various aspects of writing Airflow code, mostly demonstrated with examples using generic operators such as the `BashOperator` and `PythonOperator`. While these operators can run arbitrary code and thus could run any workload, the Airflow project also holds other operators for more specific use cases, for example, running a query on a Postgres database. These operators have one and only one specific use case, such as running a query. As a result, they are easy to use by simply providing the query to the operator, and the operator internally handles the querying logic. With a `PythonOperator`, you would have to write such querying logic yourself.

For the record, with the phrase *external system* we mean any technology other than Airflow and the machine Airflow is running on. This could be, for example,

Microsoft Azure Blob Storage, an Apache Spark cluster, or a Google BigQuery data warehouse.

To see when and how to use such operators, in this chapter we'll develop two DAGs connecting to external systems and moving and transforming data between these systems. We will inspect the various options Airflow holds (and does not hold)[1] to deal with this use case and the external systems.

In section 7.1, we develop a machine learning model on AWS, working with AWS S3 buckets and AWS SageMaker, a solution for developing and deploying machine learning models. Next, in section 7.2, we demonstrate how to move data between various systems with a Postgres database containing Airbnb places to stay in Amsterdam. The data comes from Inside Airbnb (http://insideairbnb.com), a website and public data managed by Airbnb, with records about listings, reviews, and more. Once a day we will download the latest data from the Postgres database into our AWS S3 bucket. From there, we will run a Pandas job inside a Docker container to determine the price fluctuations, and the result is saved back to S3.

7.1 Connecting to cloud services

A large portion of software runs on cloud services nowadays. Such services can generally be controlled via an API—an interface to connect and send requests to your cloud provider. The API typically comes with a client in the form of a Python package, for example, AWS's client is named boto3 (https://github.com/boto/boto3), GCP's client is named the Cloud SDK (https://cloud.google.com/sdk), and Azure's client is appropriately named the Azure SDK for Python (https://docs.microsoft.com/azure/python). Such clients provide convenient functions where, bluntly said, you enter the required details for a request and the clients handle the technical internals of handling the request and response.

In the context of Airflow, to the programmer the interface is an operator. Operators are the convenience classes to which you can provide the required details to make a request to a cloud service, and the operator internally handles the technical implementation. These operators internally make use of the Cloud SDK to send requests and provide a small layer around the Cloud SDK, which provides certain functionality to the programmer (figure 7.1).

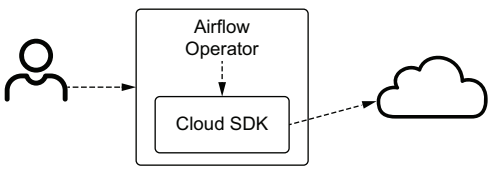


Figure 7.1 An Airflow operator translates given arguments to operations on the Cloud SDK.

[1] Operators are always under development. This chapter was written in 2020; please note at the time of reading there might be new operators that suit your use case that were not described in this chapter.

7.1.1 Installing extra dependencies

The `apache-airflow` Python package includes a few essential operators but no components to connect with any cloud. For the cloud services, you can install the provider packages in table 7.1.

Table 7.1 Extra packages to install for additional cloud provider Airflow components

Cloud	Pip install command
AWS	pip install `apache-airflow-providers-amazon`
GCP	pip install `apache-airflow-providers-google`
Azure	pip install `apache-airflow-providers-microsoft-azure`

This goes not only for the cloud providers but also for other external services. For example, to install operators and corresponding dependencies required for running the `PostgresOperator`, install the `apache-airflow-providers-postgres` package. For a full list of all available additional packages, refer to the Airflow documentation (https://airflow.apache.org/docs).

Let's look at an operator to perform an action on AWS. Take, for example, the `S3CopyObjectOperator`. This operator copies an object from one bucket to another. It accepts several arguments (skipping the irrelevant arguments for this example).

Listing 7.1 The S3CopyObjectOperator only requires you to fill the necessary details

```
➥ from airflow.providers.amazon.aws.operators.s3_copy_object import
    S3CopyObjectOperator

S3CopyObjectOperator(
    task_id="...",
    source_bucket_name="databucket",
    source_bucket_key="/data/{{ ds }}.json",
    dest_bucket_name="backupbucket",
    dest_bucket_key="/data/{{ ds }}-backup.json",
)
```

The bucket to copy from

The object name to copy

The bucket to copy to

The target object name

This operator makes copying an object on S3 to a different location on S3 a simple exercise of filling in the blanks, without needing to dive into the details of AWS's boto3 client.[2]

7.1.2 Developing a machine learning model

Let's look at a more complex example and work with a number of AWS operators by developing a data pipeline building a handwritten numbers classifier. The model will be trained on the MNIST (http://yann.lecun.com/exdb/mnist) data set, containing approximately 70,000 handwritten digits 0–9 (figure 7.2).

[2] If you check the implementation of the operator, internally it calls `copy_object()` on boto3.

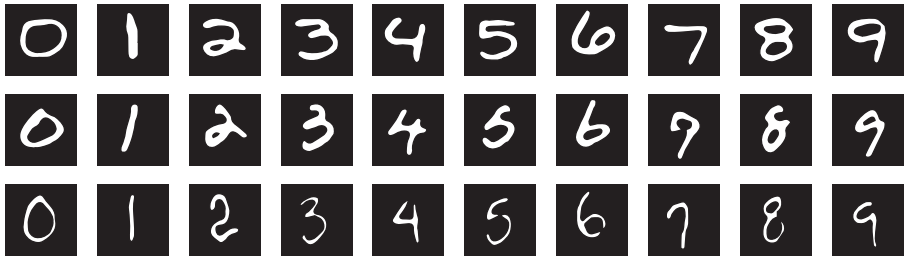


Figure 7.2 Example handwritten digits in the MNIST data set

After training the model, we should be able to feed it a new, previously unseen, handwritten number, and the model should classify the handwritten number (figure 7.3).

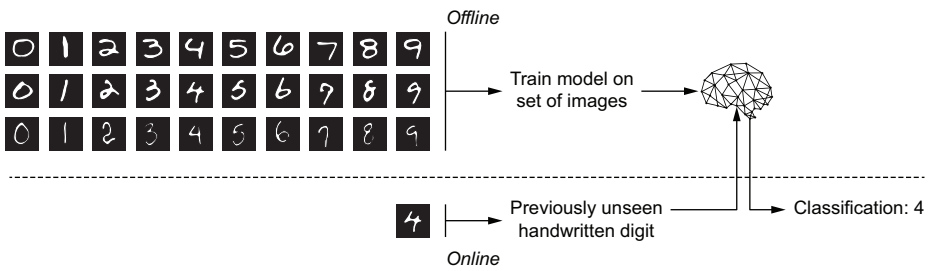


Figure 7.3 Rough outline of how a machine learning model is trained in one stage and classifies previously unseen samples in another

There are two parts to the model: an offline and an online part. The offline part takes a large set of handwritten digits, trains a model to classify these handwritten digits, and the result (a set of model parameters) is stored. This process can be done periodically when new data is collected. The online part is responsible for loading the model and classifying previously unseen digits. This should run instantaneously, as users expect direct feedback.

Airflow workflows are typically responsible for the offline part of a model. Training a model comprises data loading, preprocessing it into a format suitable for the model, and training the model, which can become complex. Also, periodically retraining the model fits nicely with Airflow's batch-processing paradigm. The online part is typically an API, such as a REST API or HTML page with REST API calls under the hood. Such an API is typically deployed only once or as part of a CI/CD pipeline. There is no use case for redeploying an API every week, and therefore it's typically not part of an Airflow workflow.

For training a handwritten digit classifier, we'll develop an Airflow pipeline. The pipeline will use AWS SageMaker, an AWS service facilitating the development and deployment of machine learning models. In the pipeline, we first copy sample data from a public location to our own S3 bucket. Next, we transform the data into a format usable for the model, train the model with AWS SageMaker, and finally, deploy the model to classify a given handwritten digit. The pipeline will look figure 7.4.

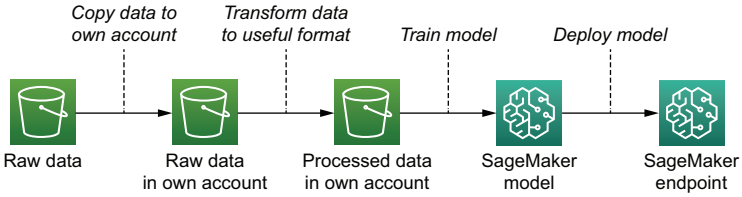


Figure 7.4 Logical steps to create a handwritten digit classifier

The depicted pipeline could run just once and the SageMaker model could be deployed just once. The strength of Airflow is the ability to schedule such a pipeline and rerun (partial) pipelines if desired in case of new data or changes to the model. If the raw data is continuously updated, the Airflow pipeline will periodically reload the raw data and redeploy the model, trained on the new data. Also, a data scientist could tune the model to their liking, and the Airflow pipeline could automatically redeploy the model without having to manually trigger anything.

Airflow holds several operators on various services of the AWS platform. While the list is never complete because services are continuously added, changed, or removed, most AWS services are supported by an Airflow operator. AWS operators are provided by the `apache-airflow-providers-amazon` package.

Let's look at the pipeline (figure 7.5).

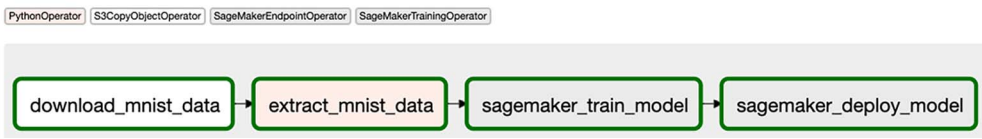


Figure 7.5 Logical steps implemented in Airflow DAG

Even though there are just four tasks, there's quite a lot to configure on AWS SageMaker, and hence the DAG code is lengthy. No worries, though; we'll break it down afterward.

Listing 7.2 DAG to train and deploy a handwritten digit classifier

```
import gzip
import io
import pickle

import airflow.utils.dates
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from airflow.providers.amazon.aws.operators.s3_copy_object import
    S3CopyObjectOperator
from airflow.providers.amazon.aws.operators.sagemaker_endpoint import
    SageMakerEndpointOperator
from airflow.providers.amazon.aws.operators.sagemaker_training import
    SageMakerTrainingOperator
from sagemaker.amazon.common import write_numpy_to_dense_tensor

dag = DAG(
   dag_id="chapter7_aws_handwritten_digits_classifier",
   schedule_interval=None,
   start_date=airflow.utils.dates.days_ago(3),
)

download_mnist_data = S3CopyObjectOperator(
   task_id="download_mnist_data",
   source_bucket_name="sagemaker-sample-data-eu-west-1",
   source_bucket_key="algorithms/kmeans/mnist/mnist.pkl.gz",
   dest_bucket_name="[your-bucket]",
   dest_bucket_key="mnist.pkl.gz",
   dag=dag,
)


def _extract_mnist_data():
   s3hook = S3Hook()

   # Download S3 dataset into memory
   mnist_buffer = io.BytesIO()
   mnist_obj = s3hook.get_key(
       bucket_name="[your-bucket]",
       key="mnist.pkl.gz",
   )
   mnist_obj.download_fileobj(mnist_buffer)

   # Unpack gzip file, extract dataset, convert, upload back to S3
   mnist_buffer.seek(0)
   with gzip.GzipFile(fileobj=mnist_buffer, mode="rb") as f:
       train_set, _, _ = pickle.loads(f.read(), encoding="latin1")
       output_buffer = io.BytesIO()
       write_numpy_to_dense_tensor(
           file=output_buffer,
           array=train_set[0],
           labels=train_set[1],
       )
```

The S3CopyObjectOperator copies objects between two S3 locations.

Sometimes your desired functionality is not supported by any operator and you have to implement the logic yourself.

We can use the S3Hook for operations on S3.

Download the S3 object.

```
        output_buffer.seek(0)
        s3hook.load_file_obj(
            output_buffer,
            key="mnist_data",
            bucket_name="[your-bucket]",
            replace=True,
        )

extract_mnist_data = PythonOperator(
    task_id="extract_mnist_data",
    python_callable=_extract_mnist_data,
    dag=dag,
)

sagemaker_train_model = SageMakerTrainingOperator(
    task_id="sagemaker_train_model",
    config={
        ➥ "TrainingJobName": "mnistclassifier-{{ execution_date.strftime('%Y-
      %m-%d-%H-%M-%S') }}",
        "AlgorithmSpecification": {
            ➥ "TrainingImage": "438346466558.dkr.ecr.eu-west-
      1.amazonaws.com/kmeans:1",
            "TrainingInputMode": "File",
        },
        "HyperParameters": {"k": "10", "feature_dim": "784"},
        "InputDataConfig": [
            {
                "ChannelName": "train",
                "DataSource": {
                    "S3DataSource": {
                        "S3DataType": "S3Prefix",
                        "S3Uri": "s3://[your-bucket]/mnist_data",
                        "S3DataDistributionType": "FullyReplicated",
                    }
                },
            }
        ],
        ➥ "OutputDataConfig": {"S3OutputPath": "s3://[your-bucket]/
      mnistclassifier-output"},
        "ResourceConfig": {
            "InstanceType": "ml.c4.xlarge",
            "InstanceCount": 1,
            "VolumeSizeInGB": 10,
        },
        ➥ "RoleArn": "arn:aws:iam::297623009465:role/service-role/
      AmazonSageMaker-ExecutionRole-20180905T153196",
        "StoppingCondition": {"MaxRuntimeInSeconds": 24 * 60 * 60},
    },
    wait_for_completion=True,
    print_log=True,
    check_interval=10,
    dag=dag,
)
```

Upload the extracted data back to S3.

Sometimes your desired functionality is not supported by any operator and you have to implement it yourself and call with the PythonOperator.

The SageMakerTraining-Operator creates a Sage-Maker training job.

The config is a JSON holding the training job configuration.

The operator conveniently waits until the training job is completed and prints CloudWatch logs while training.

```
sagemaker_deploy_model = SageMakerEndpointOperator(
    task_id="sagemaker_deploy_model",
    wait_for_completion=True,
    config={
        "Model": {
            ➥ "ModelName": "mnistclassifier-{{ execution_date.strftime('%Y-
      %m-%d-%H-%M-%S') }}",
            "PrimaryContainer": {
                ➥ "Image": "438346466558.dkr.ecr.eu-west-1.amazonaws.com/
      kmeans:1",
                "ModelDataUrl": (
                    "s3://[your-bucket]/mnistclassifier-output/"
                    ➥ "mnistclassifier-{{ execution_date.strftime('%Y-%m-%d-
      %H-%M-%S') }}/"
                    "output/model.tar.gz"
                ),  # this will link the model and the training job
            },
            ➥ "ExecutionRoleArn": "arn:aws:iam::297623009465:role/service-
      role/AmazonSageMaker-ExecutionRole-20180905T153196",
        },
        "EndpointConfig": {
            ➥ "EndpointConfigName": "mnistclassifier-{{
      execution_date.strftime('%Y-%m-%d-%H-%M-%S') }}",
            "ProductionVariants": [
                {
                    "InitialInstanceCount": 1,
                    "InstanceType": "ml.t2.medium",
                    "ModelName": "mnistclassifier",
                    "VariantName": "AllTraffic",
                }
            ],
        },
        "Endpoint": {
            ➥ "EndpointConfigName": "mnistclassifier-{{
      execution_date.strftime('%Y-%m-%d-%H-%M-%S') }}",
            "EndpointName": "mnistclassifier",
        },
    },
    dag=dag,
)

➥ download_mnist_data >> extract_mnist_data >> sagemaker_train_model >>
    sagemaker_deploy_model
```

The SageMakerEndpointOperator deploys the trained model, which makes it available behind an HTTP endpoint.

With external services, the complexity often does not lie within Airflow but with ensuring the correct integration of various components in your pipeline. There's quite a lot of configuration involved with SageMaker, so let's break down the tasks piece by piece.

Listing 7.3 Copying data between two S3 buckets

```
download_mnist_data = S3CopyObjectOperator(
    task_id="download_mnist_data",
    source_bucket_name="sagemaker-sample-data-eu-west-1",
```

```
    source_bucket_key="algorithms/kmeans/mnist/mnist.pkl.gz",
    dest_bucket_name="[your-bucket]",
    dest_bucket_key="mnist.pkl.gz",
    dag=dag,
)
```

After initializing the DAG, the first task copies the MNIST data set from a public bucket to our own bucket. We store it in our own bucket for further processing. The `S3CopyObjectOperator` asks for both the bucket and object name on the source and destination and will copy the selected object for you. So, while developing, how do we verify this works correctly, without first coding the full pipeline and keeping fingers crossed to see if it works in production?

7.1.3 Developing locally with external systems

Specifically for AWS, if you have access to the cloud resources from your development machine with an access key, you can run Airflow tasks locally. With the help of the CLI command `airflow tasks test`, we can run a single task for a given execution date. Since the `download_mnist_data` task doesn't use the execution date, it doesn't matter what value we provide. However, say the `dest_bucket_key` was given as `mnist-{{ ds }}.pkl.gz`; then we'd have to think wisely about what execution date we test with. From your command line, complete the steps in the following listing.

Listing 7.4 Setting up for locally testing AWS operators

```
# Add secrets in ~/.aws/credentials:
  # [myaws]
  # aws_access_key_id=AKIAEXAMPLE123456789
  # aws_secret_access_key=supersecretaccesskeydonotshare!123456789

export AWS_PROFILE=myaws
export AWS_DEFAULT_REGION=eu-west-1
export AIRFLOW_HOME=[your project dir]
airflow db init
airflow tasks test chapter7_aws_handwritten_digits_classifier
     download_mnist_data 2020-01-01
```

Initialize a local Airflow metastore.

Run a single task.

This will run the task `download_mnist_data` and display logs.

Listing 7.5 Verifying a task manually with airflow tasks test

```
➥ $ airflow tasks test chapter7_aws_handwritten_digits_classifier
     download_mnist_data 2019-01-01

INFO - Using executor SequentialExecutor
INFO - Filling up the DagBag from .../dags
➥ INFO - Dependencies all met for <TaskInstance:
     chapter7_aws_handwritten_digits_classifier.download_mnist_data 2019-01-
     01T00:00:00+00:00 [None]>
--------------------------------------------------------------------------------
```

```
INFO - Starting attempt 1 of 1
--------------------------------------------------------------------------------
➥ INFO - Executing <Task(PythonOperator): download_mnist_data> on 2019-01-
    01T00:00:00+00:00
INFO - Found credentials in shared credentials file: ~/.aws/credentials
INFO - Done. Returned value was: None
➥ INFO - Marking task as SUCCESS.dag_id=chapter7_aws_handwritten_digits
    _classifier, task_id=download_mnist_data, execution_date=20190101T000000,
    start_date=20200208T110436, end_date=20200208T110541
```

After this, we can see the data was copied into our own bucket (figure 7.6).

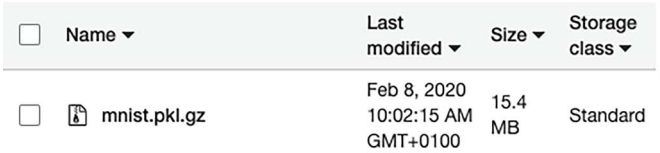


Figure 7.6 After running the task locally with `airflow tasks test`, the data is copied to our own AWS S3 bucket.

What just happened? We configured the AWS credentials to allow us to access the cloud resources from our local machine. While this is specific to AWS, similar authentication methods apply to GCP and Azure. The AWS boto3 client used internally in Airflow operators will search in various places for credentials on the machine where a task is run. In listing 7.4, we set the `AWS_PROFILE` environment variable, which the boto3 client picks up for authentication. After this, we set another environment variable: `AIRFLOW_HOME`. This is the location where Airflow will store logs and such. Inside this directory, Airflow will search for a /dags directory. If that happens to live elsewhere, you can point Airflow there with another environment variable: `AIRFLOW__CORE__DAGS_FOLDER`.

Next, we run `airflow db init`. Before doing this, ensure you either have not set `AIRFLOW__CORE__SQL_ALCHEMY_CONN` (a URI that points to a database for storing all state), or set it to a database URI specifically for testing purposes. Without `AIRFLOW__CORE__SQL_ALCHEMY_CONN` set, `airflow db init` initializes a local SQLite database (a single file, no configuration required, database) inside `AIRFLOW_HOME`.[3] `airflow tasks test` exists for running and verifying a single task and does not record any state in the database; however, it does require a database for storing logs, and therefore we must initialize one with `airflow db init`.

After all this, we can run the task from the command line with `airflow tasks test chapter7_aws_handwritten_digits_classifier extract_mnist_data 2020-01-01`.

[3] The database will be generated in a file name airflow.db in the directory set by `AIRFLOW_HOME`. You can open and inspect it with, for example, DBeaver.

After we've copied the file to our own S3 bucket, we need to transform it into a format the SageMaker KMeans model expects, which is the RecordIO format.[4]

Listing 7.6 Transforming MNIST data to RecordIO format for SageMaker KMeans model

```
import gzip
import io
import pickle

from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from sagemaker.amazon.common import write_numpy_to_dense_tensor


def _extract_mnist_data():
    s3hook = S3Hook()

    # Download S3 dataset into memory
    mnist_buffer = io.BytesIO()
    mnist_obj = s3hook.get_key(
        bucket_name="your-bucket",
        key="mnist.pkl.gz",
    )
    mnist_obj.download_fileobj(mnist_buffer)

    # Unpack gzip file, extract dataset, convert, upload back to S3
    mnist_buffer.seek(0)
    with gzip.GzipFile(fileobj=mnist_buffer, mode="rb") as f:
        train_set, _, _ = pickle.loads(f.read(), encoding="latin1")
        output_buffer = io.BytesIO()
        write_numpy_to_dense_tensor(
            file=output_buffer,
            array=train_set[0],
            labels=train_set[1],
        )
        output_buffer.seek(0)
        s3hook.load_file_obj(
            output_buffer,
            key="mnist_data",
            bucket_name="your-bucket",
            replace=True,
        )


extract_mnist_data = PythonOperator(
    task_id="extract_mnist_data",
    python_callable=_extract_mnist_data,
    dag=dag,
)
```

Initialize S3Hook to communicate with S3.

Download data into in-memory binary stream.

Unzip and unpickle

Convert Numpy array to RecordIO records.

Upload result to S3.

[4] Mime type `application/x-recordio-protobuf` documentation: https://docs.aws.amazon.com/sagemaker/latest/dg/cdf-inference.html.

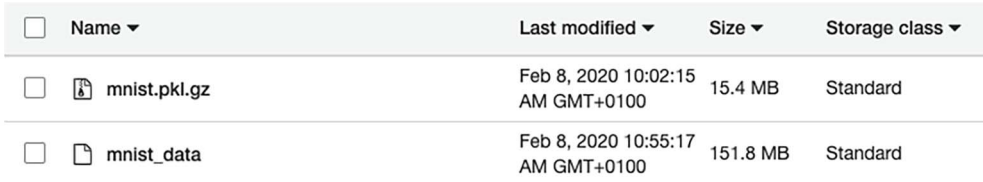
Airflow in itself is a general-purpose orchestration framework with a manageable set of features to learn. However, working in the data field often takes time and experience to know about all technologies and to know which dots to connect in which way. You never develop Airflow alone; oftentimes you're connecting to other systems and reading the documentation for that specific system. While Airflow will trigger the job for such a task, the difficulty in developing a data pipeline often lies outside Airflow and with the system that you're communicating with. While this book focuses solely on Airflow, due to the nature of working with other data-processing tools, we try to demonstrate, via these examples, what it's like to develop a data pipeline.

For this task, there is no existing functionality in Airflow for downloading data and extracting, transforming, and uploading the result back to S3. Therefore, we must implement our own function. The function downloads the data into an in-memory binary stream (io.BytesIO) so that the data is never stored in a file on the filesystem and so that no remaining files are left after the task. The MNIST data set is small (15 MB) and will therefore run on any machine. However, think wisely about the implementation; for larger data it might be wise to opt for storing the data on disks and processing in chunks.

Similarly, this task can also be run/tested locally with

```
airflow tasks test chapter7_aws_handwritten_digits_classifier extract_mnist_data
     2020-01-01
```

Once completed, the data will be visible in S3 (figure 7.7).



Name	Last modified	Size	Storage class
mnist.pkl.gz	Feb 8, 2020 10:02:15 AM GMT+0100	15.4 MB	Standard
mnist_data	Feb 8, 2020 10:55:17 AM GMT+0100	151.8 MB	Standard

Figure 7.7 Gzipped and pickled data was read and transformed into a usable format.

The next two tasks train and deploy the SageMaker model. The SageMaker operators take a config argument, which entails configuration specific to SageMaker and is not in the scope of this book. Let's focus on the other arguments.

Listing 7.7 Training an AWS SageMaker model

```
sagemaker_train_model = SageMakerTrainingOperator(
   task_id="sagemaker_train_model",
   config={
      "TrainingJobName": "mnistclassifier-{{ execution_date.strftime('%Y-
     %m-%d-%H-%M-%S') }}",
```

```
        ...
    },
    wait_for_completion=True,
    print_log=True,
    check_interval=10,
    dag=dag,
)
```

Many of the details in config are specific to SageMaker and can be discovered by reading the SageMaker documentation. Two lessons applicable to working with any external system can be made, though.

First, AWS restricts the `TrainingJobName` to be unique within an AWS account and region. Running this operator with the same `TrainingJobName` twice will return an error. Say we provided a fixed value, `mnistclassifier`, to the `TrainingJobName`; running it a second time would result in failure:

```
botocore.errorfactory.ResourceInUse: An error occurred (ResourceInUse) when
calling the CreateTrainingJob operation: Training job names must be unique
within an AWS account and region, and a training job with this name already
exists (arn:aws:sagemaker:eu-west-1:[account]:training-job/mnistclassifier)
```

The config argument can be templated, and, hence, if you plan to retrain your model periodically, you must provide it a unique `TrainingJobName`, which we can do by templating it with the `execution_date`. This way we ensure our task is idempotent and existing training jobs do not result in conflicting names.

Second, note the arguments `wait_for_completion` and `check_interval`. If `wait_for_completion` were set to false, the command would simply *fire and forget* (that's how the boto3 client works): AWS would start a training job, but we'd never know if the training job completed successfully. Therefore, all SageMaker operators wait (default `wait_for_completion=True`) for the given task to complete. Internally, the operators poll every X seconds, checking to see if the job is still running. This ensures our Airflow tasks only complete once finished (figure 7.8). If you have downstream tasks and want to ensure the correct behavior and order of your pipeline, you'll want to wait for completion.

Figure 7.8 The SageMaker operators only succeed once the job is completed successfully in AWS.

Once the full pipeline is complete, we have successfully deployed a SageMaker model and endpoint to expose it (figure 7.9).

However, in AWS, a SageMaker endpoint is not exposed to the outside world. It is accessible via the AWS APIs, but not, for example, via a worldwide accessible HTTP

	Name	ARN	Creation time	Status	Last updated
○	mnistclassifier	arn:aws:sagemaker:eu-west-1:[accountid]:endpoint/mnistclassifier	Feb 07, 2020 12:15 UTC	InService	Feb 09, 2020 12:47 UTC

Figure 7.9 In the SageMaker model menu, we can see the model was deployed and the endpoint is operational.

endpoint. Of course, to complete the data pipeline we'd like to have a nice interface or API to feed handwritten digits and receive a result. In AWS, in order to make it accessible to the internet, we could deploy a lambda (https://aws.amazon.com/lambda) to trigger the SageMaker endpoint and an API gateway (https://aws.amazon.com/api-gateway) to create an HTTP endpoint, forwarding requests to the Lambda,[5] so why not integrate them into our pipeline (figure 7.10)?

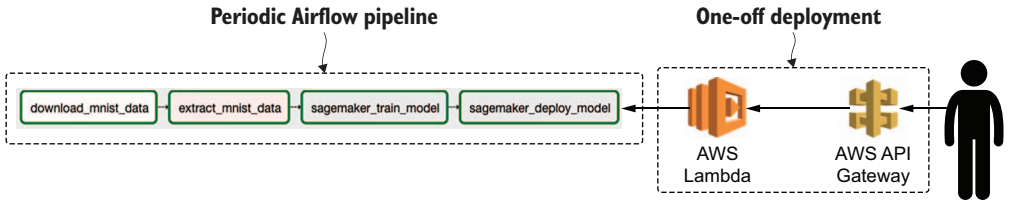


Figure 7.10 The handwritten digit classifier exists of more components than just the Airflow pipeline.

The reason for not deploying infrastructure is the fact the Lambda and API Gateway will be deployed as one-offs, not periodically. They operate in the online stage of the model and therefore are better deployed as part of a CI/CD pipeline. For the sake of completeness, the API can be implemented with Chalice.

Listing 7.8 An example user-facing API using AWS Chalice

```
import json
from io import BytesIO

import boto3
import numpy as np
from PIL import Image
from chalice import Chalice, Response
from sagemaker.amazon.common import numpy_to_record_serializer

app = Chalice(app_name="number-classifier")
```

[5] Chalice (https://github.com/aws/chalice) is a Python framework similar to Flask for developing an API and automatically generating the underlying API gateway and lambda resources in AWS.

```
@app.route("/", methods=["POST"], content_types=["image/jpeg"])
def predict():
    """
    Provide this endpoint an image in jpeg format.
    The image should be equal in size to the training images (28x28).
    """
    img = Image.open(BytesIO(app.current_request.raw_body)).convert("L")
    img_arr = np.array(img, dtype=np.float32)
    runtime = boto3.Session().client(
        service_name="sagemaker-runtime",
        region_name="eu-west-1",
    )
    response = runtime.invoke_endpoint(
        EndpointName="mnistclassifier",
        ContentType="application/x-recordio-protobuf",
        Body=numpy_to_record_serializer()(img_arr.flatten()),
    )
    result = json.loads(response["Body"].read().decode("utf-8"))
    return Response(
        result,
        status_code=200,
        headers={"Content-Type": "application/json"},
)
```

Convert input image to grayscale numpy array.

Invoke the SageMaker endpoint deployed by the Airflow DAG.

The SageMaker response is returned as bytes.

The API holds one single endpoint, which accepts a JPEG image.

Listing 7.9 Classifying a handwritten image by submitting it to the API

```
curl --request POST \
  --url http://localhost:8000/ \
  --header 'content-type: image/jpeg' \
  --data-binary @'/path/to/image.jpeg'
```

The result, if trained correctly, looks like figure 7.11.

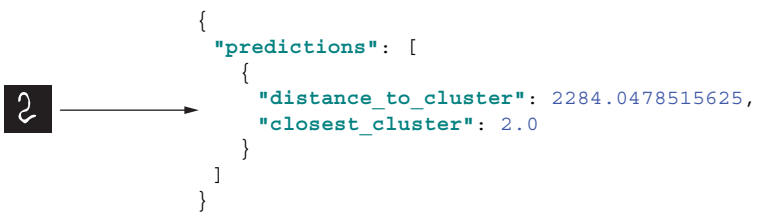


Figure 7.11 Example API input and output. A real product could display a nice UI for uploading images and displaying the predicted number.

The API transforms the given image into RecordIO format, just like the SageMaker model was trained on. The RecordIO object is then forwarded to the SageMaker endpoint deployed by the Airflow pipeline, and finally returns a prediction for the given image.

7.2 Moving data from between systems

A classic use case for Airflow is a periodic ETL job, where data is downloaded daily and transformed elsewhere. Such a job is often for analytical purposes, where data is exported from a production database and stored elsewhere for processing later. The production database is most often (depending on the data model) not capable of returning historical data (e.g., the state of the database as it was one month ago). Therefore, a periodic export is often created and stored for later processing. Historic data dumps will grow your storage requirements quickly and require distributed processing to crunch all data. In this section, we'll explore how to orchestrate such a task with Airflow.

We developed a GitHub repository with code examples to go along with this book. It contains a Docker Compose file for deploying and running the next use case, where we extract Airbnb listings data and process it in a Docker container with Pandas. In a large-scale data processing job, the Docker container could be replaced by a Spark job, which distributes the work over multiple machines. The Docker Compose file contains the following:

- One Postgres container holding the Airbnb Amsterdam listings.
- One AWS S3-API-compatible container. Since there is no AWS S3-in-Docker, we created a MinIO container (AWS S3 API compatible object storage) for reading/writing data.
- One Airflow container.

Visually, the flow will look like figure 7.12.

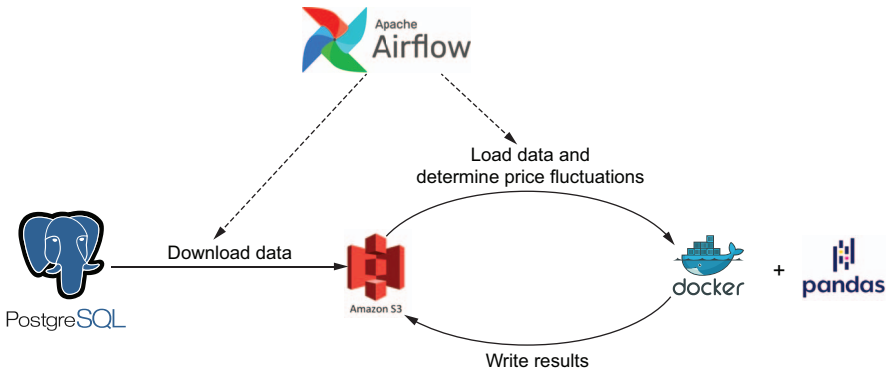


Figure 7.12 Airflow managing jobs moving data between various systems

Airflow acts as the "spider in the web," starting and managing jobs and ensuring all finish successfully in the correct order, failing the pipeline if not.

The Postgres container is a custom-built Postgres image holding a database filled with Inside Airbnb data, available on Docker Hub as airflowbook/insideairbnb. The database holds one single table named "listings," which contains records of places in Amsterdam listed on Airbnb between April 2015 and December 2019 (figure 7.13).

Figure 7.13 Table structure of example Inside Airbnb database

Let's first query the database and export data to S3. From there we will read and process the data with Pandas.

A common task in Airflow is a data transfer between two systems, possibly with a transformation in between. Querying a MySQL database and storing the result on Google Cloud Storage, copying data from an SFTP server to your data lake on AWS S3, or calling an HTTP REST API and storing the output have one thing in common, namely that they deal with two systems: one for the input and one for the output.

In the Airflow ecosystem, this has led to the development of many such A-to-B operators. For these examples, we have the `MySqlToGoogleCloudStorageOperator`, `SFTPToS3Operator`, and the `SimpleHttpOperator`. While there are many use cases to cover with the operators in the Airflow ecosystem, there is no `Postgres-query-to-AWS-S3` operator (at the time of writing this book). So, what to do?

7.2.1 Implementing a PostgresToS3Operator

First, we could take note of how other similar operators work and develop our own `PostgresToS3Operator`. Let's look at an operator closely related to our use case, the `MongoToS3Operator` in `airflow.providers.amazon.aws.transfers.mongo_to_s3` (after installing `apache-airflow-providers-amazon`). This operator runs a query on a MongoDB database and stores the result in an AWS S3 bucket. Let's inspect it and figure out how to replace MongoDB with Postgres. The `execute()` method is implemented as follows (some code was obfuscated).

Listing 7.10 Implementation of the MongoToS3Operator

```
def execute(self, context):
    s3_conn = S3Hook(self.s3_conn_id)

    results = MongoHook(self.mongo_conn_id).find(
        mongo_collection=self.mongo_collection,
        query=self.mongo_query,
        mongo_db=self.mongo_db
    )
```

An S3Hook is instantiated.

A MongoHook is instantiated and used to query for data.

```
    docs_str = self._stringify(self.transform(results))

    # Load Into S3
    s3_conn.load_string(
        string_data=docs_str,
        key=self.s3_key,
        bucket_name=self.s3_bucket,
        replace=self.replace
    )
```

Results are transformed.

load_string() is called on the S3Hook to write the transformed results.

It's important to note that this operator does not use any of the filesystems on the Airflow machine but keeps all results in memory. The flow is basically

```
MongoDB → Airflow in operator memory → AWS S3.
```

Since this operator keeps the intermediate results in memory, think wisely about the memory implications when running very large queries, because a very large result could potentially drain the available memory on the Airflow machine. For now, let's keep the `MongoToS3Operator` implementation in mind and look at one other A-to-B operator, the `S3ToSFTPOperator`.

Listing 7.11 Implementation of the S3ToSFTPOperator

```
def execute(self, context):
    ssh_hook = SSHHook(ssh_conn_id=self.sftp_conn_id)
    s3_hook = S3Hook(self.s3_conn_id)

    s3_client = s3_hook.get_conn()
    sftp_client = ssh_hook.get_conn().open_sftp()

    with NamedTemporaryFile("w") as f:
        s3_client.download_file(self.s3_bucket, self.s3_key, f.name)
        sftp_client.put(f.name, self.sftp_path)
```

NamedTemporaryFile is used for temporarily downloading a file, which is removed after the context exits.

This operator, again, instantiates two hooks: `SSHHook` (SFTP is FTP over SSH) and `S3Hook`. However, in this operator, the intermediate result is written to a `NamedTemporaryFile`, which is a temporary place on the local filesystem of the Airflow instance. In this situation, we do not keep the entire result in memory, but we must ensure enough disk space is available.

Both operators have two hooks in common: one for communicating with system A and one for communicating with system B. However, how data is retrieved and transferred between systems A and B is different and up to the person implementing the specific operator. In the specific case of Postgres, database cursors can iterate to fetch and upload chunks of results. However, this implementation detail is not in the scope of this book. Keep it simple and assume the intermediate result fits within the resource boundaries of the Airflow instance.

A very minimal implementation of a `PostgresToS3Operator` could look as follows.

Listing 7.12 Example implementation of a PostgresToS3Operator

```
def execute(self, context):
    postgres_hook = PostgresHook(postgres_conn_id=self._postgres_conn_id)
    s3_hook = S3Hook(aws_conn_id=self._s3_conn_id)

    results = postgres_hook.get_records(self._query)
    s3_hook.load_string(
        string_data=str(results),
        bucket_name=self._s3_bucket,
        key=self._s3_key,
    )
```

Fetch records from the PostgreSQL database.

Upload records to S3 object.

Let's inspect this code. The initialization of both hooks is straightforward; we initialize them, providing the name of the connection ID the user provides. While it is not necessary to use keyword arguments, you might notice the `S3Hook` takes the argument `aws_conn_id` (and not `s3_conn_id` as you might expect). During the development of such an operator, and the usage of such hooks, it is inevitable to sometimes dive into the source code or carefully read the documentation to view all available arguments and understand how things are propagated into classes. In the case of the `S3Hook`, it subclasses the `AwsHook` and inherits several methods and attributes, such as the `aws_conn_id`.

The `PostgresHook` is also a subclass, namely of the `DbApiHook`. By doing so, it inherits several methods, such as `get_records()`, which executes a given query and returns the results. The return type is a sequence of sequences (more precisely a list of tuples[6]). We then *stringify* the results and call `load_string()`, which writes encoded data to the given bucket/key on AWS S3. You might think this is not very practical, and you are correct. Although this is a minimal flow to run a query on Postgres and write the result to AWS S3, the list of tuples is stringified, which no data processing framework is able to interpret as an ordinary file format such as CSV or JSON (figure 7.14).

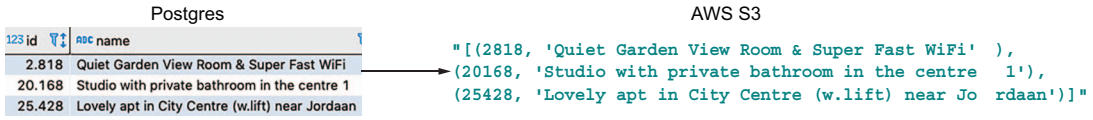


Figure 7.14 Exporting data from a Postgres database to stringified tuples

The tricky part of developing data pipelines is often not the orchestration of jobs with Airflow, but ensuring all bits and pieces of various jobs are configured correctly and fit together like puzzle pieces. So, let's write the results to CSV; this will allow

[6] As specified in PEP 249, the Python Database API Specification.

data-processing frameworks such as Apache Pandas and Spark to easily interpret the output data.

For uploading data to S3, the `S3Hook` provides various convenience methods. For file-like objects,[7] we can apply `load_file_obj()`.

Listing 7.13 In-memory conversion of Postgres query results to CSV and upload to S3

```
def execute(self, context):
    postgres_hook = PostgresHook(postgres_conn_id=self._postgres_conn_id)
    s3_hook = S3Hook(aws_conn_id=self._s3_conn_id)


    results = postgres_hook.get_records(self.query)


    data_buffer = io.StringIO()
    csv_writer = csv.writer(data_buffer, lineterminator=os.linesep)
    csv_writer.writerows(results)
    data_buffer_binary = io.BytesIO(data_buffer.getvalue().encode())
    s3_hook.load_file_obj(
        file_obj=data_buffer_binary,
        bucket_name=self._s3_bucket,
        key=self._s3_key,
        replace=True,
    )
```

For convenience, we first create a string buffer, which is like a file in memory to which we can write strings. After writing, we convert it to binary.

It requires a file-like object in binary mode.

Ensure idempotency by replacing files if they already exist.

Buffers live in memory, which can be convenient because memory leaves no remaining files on the filesystem after processing. However, we have to realize that the output of the Postgres query must fit into memory. The key to idempotency is setting `replace=True`. This ensures existing files are overwritten. We can rerun our pipeline after, for example, a code change, and then the pipeline will fail without `replace=True` because of the existing file.

With these few extra lines, we can now store CSV files on S3. Let's see it in practice.

Listing 7.14 Running the PostgresToS3Operator

```
download_from_postgres = PostgresToS3Operator(
    task_id="download_from_postgres",
    postgres_conn_id="inside_airbnb",
    query="SELECT * FROM listings WHERE download_date={{ ds }}",
    s3_conn_id="s3",
    s3_bucket="inside_airbnb",
    s3_key="listing-{{ ds }}.csv",
    dag=dag,
)
```

With this code, we now have a convenient operator that makes querying Postgres and writing the result to CSV on S3 an exercise of filling in the blanks.

[7] In-memory objects with file-operation methods for reading/writing.

7.2.2 Outsourcing the heavy work

A common discussion in the Airflow community is whether to view Airflow as not only a task orchestration system but also a task execution system since many DAGs are written with the `BashOperator` and `PythonOperator`, which execute work within the same Python runtime as Airflow. Opponents of this mindset argue for viewing Airflow only as a task-triggering system and suggest no actual work should be done inside Airflow itself. Instead, all work should be offloaded to a system intended for dealing with data, such as Apache Spark.

Let's imagine we have a very large job that would take all resources on the machine Airflow is running on. In this case, it's better to run the job elsewhere; Airflow will start the job and wait for it to complete. The idea is that there should be a strong separation between orchestration and execution, which we can achieve by Airflow starting the job and waiting for completion and a data-processing framework such as Spark performing the actual work.

In Spark, there are various ways to start a job:

- *Using the* `SparkSubmitOperator`—This requires a spark-submit binary and YARN client config on the Airflow machine to find the Spark instance.
- *Using the* `SSHOperator`—This requires SSH access to a Spark instance but does not require Spark client config on the Airflow instance.
- *Using the* `SimpleHTTPOperator`—This requires running Livy, a REST API for Apache Spark, to access Spark.

The key to working with any operator in Airflow is reading the documentation and figuring out which arguments to provide. Let's look at the `DockerOperator`, which starts the Docker container for processing the Inside Airbnb data using Pandas.

Listing 7.15 Running a Docker container with the DockerOperator

```
crunch_numbers = DockerOperator(
    task_id="crunch_numbers",
    image="airflowbook/numbercruncher",
    api_version="auto",
    auto_remove=True,
    docker_url="unix://var/run/docker.sock",
    network_mode="host",
    environment={
        "S3_ENDPOINT": "localhost:9000",
        "S3_ACCESS_KEY": "[insert access key]",
        "S3_SECRET_KEY": "[insert secret key]",
    },
    dag=dag,
)
```

Remove the container after completion.

To connect to other services on the host machine via http://localhost, we must share the host network namespace by using host network mode.

The `DockerOperator` wraps around the Python Docker client and, given a list of arguments, enables starting Docker containers. In listing 7.15, the `docker_url` is set to a Unix socket, which requires Docker running on the local machine. It starts the Docker